http-types
Hackage: https://hackage.haskell.org/package/http-types-0.12.3

```
--
module Network.HTTP.Types
type Method = ByteString                                    --
Data.ByteString i.e. strict
methodGet :: Method
methodPost :: Method
methodHead :: Method
methodPut :: Method

data StdMethod = GET | POST | HEAD | PUT | DELETE | TRACE | CONNECT
| OPTIONS | PATCH
parseMethod :: Method -> Either ByString StdMethod
renderMethod :: Either ByteString StdMethod -> Method
renderStdMethod :: StdMethod -> Method


data HttpVersion = HttpVersion {httpMajor :: Int, httpMinor :: Int}
http09 :: HttpVersion
http10 :: HttpVersion
http11 :: HttpVersion
http20 :: HttpVersion


data Status = Status {statusCode :: Int, statusMessage ::
ByteString}
status100 :: Status
status101 :: Status
... etc ...


type Header = (HeaderName, ByteString)
type HeaderName = CI ByteString                            -- CI in
Data.CaseInsensitive
type RequestHeaders = [Header]
type ResponseHeaders = [Header]
hAccept :: Header
hAuthorization :: Header
hCookie :: Header
... etc ...


type QueryItem = (ByteString, Maybe ByteString)
type Query = [Query]
type SimpleQueryItem = (ByteString, ByteString)
type SimpleQuery = [SimpleQueryItem]
simpleQueryToQuery :: SimpleQuery -> Query
renderQuery :: Bool -> Query -> ByteString
renderQueryBuilder :: Bool -> Query -> Builder
renderSimpleQuery :: Bool -> SimpleQuery -> ByteString
```

```
parseQuery :: ByteString -> Query
parseSimpleQuery :: ByteString -> SimpleQuery

-- URL
encodePathSegments :: [Text] -> Builder
decodePathSegments :: ByteString -> [Text]
encodePathSegmentsRelative :: [Text] -> Builder
extractPath :: ByteString -> ByteString
encodePath :: [Text] -> Query -> Builder
decodePath :: ByteString -> ([Text], Query)
urlEncodeBuilder :: Bool -> ByteString -> Builder
urlEncode :: Bool -> ByteString -> ByteString
urldecode :: Bool -> ByteString -> ByteString


... etc ...
```

http-api-data
Web.HttpApiData: https://hackage.haskell.org/package/http-api-data-0.5.1

```
-- Web.HttpApiData
class ToHttpApiData a where
  {-# MINIMAL toUrlPiece | toQueryParam #-}
  toQueryParam :: a -> Text
  toEncodedQueryParam :: a -> Builder
  toHeader :: a -> ByteString
  toUrlPiece :: a -> Text
  toEncodedUrlPiece :: a -> Builder

class FromHttpApiData a where
  parseUrlPiece :: Text -> Either Text a
  parseHeader :: Bytestring -> Either Text a
  parseQueryParam :: Text -> Either Text a

...
```

http-api-data
Convert Haskell type <-> HTTP data
e.g. parse query param -> Haskell type
encode Haskell type -> request query param

==========

Wai (Web application interface)

Wai (Web Application Interface)
Hackage: https://hackage.haskell.org/package/wai-3.2.3

Wai

Web application interface

```
--
module Network.Wai
type Application = Request -> (Response -> IO ResponseReceived) ->
IO ResponseReceived
type Method = ByteString

data Request
defaultRequest :: Request
data RequestBodyLength = ChunkedBody | KnownLength Word64

requestMethod :: Request -> Method
httpVersion :: Request -> HttpVersion
rawPathInfo :: Request -> ByteString
requestHeaders :: Request -> [RequestHeader]
isSecure :: Request -> Bool
remoteHost :: Request -> SockAddr
pathInfo :: Request -> [String]
queryString :: Request -> Query
getRequestBodyChunk :: Request -> IO ByteString
requestBody :: Request -> IO ByteString
requestBodyLength :: Request -> RequestBodyLength
requestBodyHeaderRange :: Request -> Maybe ByteString
requestBodyReferer :: Request -> Maybe ByteString
requestBodyUserAgent :: Request -> Maybe ByteString
strictRequestBody :: Request -> IO ByteString
lazyRequestBody :: Request -> IO ByteString
consumeRequestBody :: Request -> IO ByteString


data Response
data StreamingBody = ((Builder -> IO ()) -> IO () -> IO ()
data FilePart = FilePart {filePartOffset :: Integer,
filePartByteCount :: Integer, filePartFileSize :: Integer}
responseFile :: Status -> ResponseHeaders -> FilePath -> Maybe
FilePart -> Response
responseBuilder :: Status -> ResponseHeaders -> Builder -> Response
responseLBS :: Status -> ResponseHeaders -> ByteString -> Response
responseStream :: Status -> ResponseHeaders -> StreamingBody ->
Response
responseRaw :: (IO ByteString -> (ByteString -> IO ()) -> IO ()) ->
Response -> Response
responseStatus :: Response -> Status
responseHeaders :: Response -> ResponseHeaders
responseToStream :: Response -> (Status, ResponseHeaders,
(StreamingBody -> IO a) -> IO a)
mapResponseHeaders :: (ResponseHeader -> ResponseHeader) ->
Response -> Response


-- Middleware
```

```
type Middleware = Application -> Application
ifRequest :: (Request -> Bool) -> Middleware -> Middleware
modifyResponse :: (Response -> Response) -> Middleware
```

Example: Create log middleware

```
{-# LANGUAGE OverloadedStrings #-}

import Data.ByteString.Char8 qualified as B
import Network.HTTP.Types
import Network.Wai
import Network.Wai.Handler.Warp
import Data.Function

start :: IO
start = run 12345 $ mainApp
  & logger

mainApp :: Application
mainApp = ...

logger :: Middleware
logger app req sendResponse = app req (\res -> logRequest req res
>> sendResponse res)

logRequest :: Request -> Response -> IO ()
logRequest req res =
  do
    B.putStrLn $
      -- Request
      showb (httpVersion req)
        <> " "
        <> requestMethod req
        <> " "
        <> rawPathInfo req
        <> rawQueryString req
        <> " ["
        <> showb (requestBodyLength req)
        <> "B]"
        <> "\n "
        <> showb (requestHeaders req)
        -- Response
        <> "\n \t"
        <> "--> "
        <> showb (responseStatus res)
        <> " "
        <> showb (responseHeaders res)

showb :: Show a => a -> B.ByteString
showb = B.pack . show
```

e.g. logs

```
HTTP/1.1 GET /items?
term_type=weekly&term_start_date=2020-06-15&store_code=100119
[KnownLength 0B]
 [("Host","localhost:4010"),("Connection","keep-alive"),
("Accept","*/*"),("User-Agent","%5BDEBUG%5D%20Store%20Layout/0
CFNetwork/1402.0.8 Darwin/21.6.0"),("Accept-Language","en-US"),
("Authorization","Bearer testStoreUserRW_token"),("Accept-
Encoding","gzip, deflate")]
        --> Status {statusCode = 200, statusMessage = "OK"}
[("Content-Type","application/json;charset=utf-8")]
HTTP/1.1 GET /facings?
term_type=weekly&term_start_date=2020-06-15&store_code=100119
[KnownLength 0B]
 [("Host","localhost:4010"),("Connection","keep-alive"),
("Accept","*/*"),("User-Agent","%5BDEBUG%5D%20Store%20Layout/0
CFNetwork/1402.0.8 Darwin/21.6.0"),("Authorization","Bearer
testStoreUserRW_token"),("Accept-Language","en-US"),("Accept-
Encoding","gzip, deflate")]
        --> Status {statusCode = 200, statusMessage = "OK"}
[("Content-Type","application/json;charset=utf-8")]
HTTP/1.1 POST /layout-diff [KnownLength 199B]
 [("Host","localhost:4010"),("Content-Type","application/json"),
("User-Agent","%5BDEBUG%5D%20Store%20Layout/0 CFNetwork/1402.0.8
Darwin/21.6.0"),("Connection","keep-alive"),("Accept","*/*"),
("Accept-Language","en-US"),("Authorization","Bearer
testStoreUserRW_token"),("Accept-Encoding","gzip, deflate"),
("Content-Length","199")]
        --> Status {statusCode = 200, statusMessage = "OK"}
[("Content-Type","application/json;charset=utf-8")]
```

GOTCHA: cannot read request body twice e.g. read request in a logging middleware
Problem:

```
{-# LANGUAGE OverloadedStrings #-}

import Data.ByteString.Char8 qualified as B
import Network.HTTP.Types
import Network.Wai
import Network.Wai.Handler.Warp
import Data.Function

start :: IO
start = run 12345 $ mainApp
  & logger

mainApp :: Application
```

```
mainApp req sendResponse = do
  body <- consumeRequestBodyStrict req               -- BAD. Body
is empty since it was already read by the logger.

logger :: Middleware
logger app req f = do
  body <- consumeRequestBodyStrict req               -- Body read
by logger first
  B.putStrLn body
  app req f
```

Solution:
Ref: https://github.com/haskell-servant/servant/issues/1120
The problem happens because the request body is a **stream** which can only be consumed **once**.
Solution is to **loop** the request body back after it is consumed.

```
{-# LANGUAGE BlockArguments #-}
{-# LANGUAGE OverloadedStrings #-}
{-# OPTIONS_GHC -Wno-deprecations #-}

module Middleware.FreezeRequest
  ( freezeReqBody,
  )
where

import Control.Concurrent
import Control.Monad.IO.Class
import Control.Monad.Loops
import Data.ByteString qualified as BS
import Data.Sequence (Seq (Empty, (:<|)), (|>))
import Data.Sequence qualified as S
import Network.Wai


-- WORKAROUND:
-- It is not possible to easily read Request Body twice.
-- e.g.
--      log the request body in a middleware
--
-- Trying to read the body again fails with no request body.
-- Below is a workaround that loops the body back.
--
-- Ref: https://github.com/haskell-servant/servant/issues/
1120#issuecomment-898940489
freezeReqBody :: Middleware
freezeReqBody app req handle' = do
  body' <- newMVar =<< extractBody req
  let req' = req {requestBody = requestBody' body'}
  app req' handle'
  where
```

```
    requestBody' mvar = modifyMVar mvar \chunks -> do
      case chunks of
        (h :<| rest) -> return (rest |> h, h)
        Empty -> return (Empty, mempty)

extractBody :: MonadIO m => Request -> m (Seq BS.ByteString)
extractBody req = do
  chunk <- S.singleton <$> liftIO (getRequestBodyChunk req)
  (flip $ iterateUntilM r) chunk $ \chunks' -> do
    chunk' <- liftIO $ getRequestBodyChunk req
    return $ chunks' |> chunk'
  where
    r :: Seq BS.ByteString -> Bool
    r seq' = case S.viewr seq' of
      S.EmptyR -> True
      (_ S.:> e) -> e == ""
```

main

```
{-# LANGUAGE OverloadedStrings #-}

import Data.ByteString.Char8 qualified as B
import Network.HTTP.Types
import Network.Wai
import Network.Wai.Handler.Warp
import Data.Function
import Middleware.FreezeRequest

start :: IO
start = run 12345 $ mainApp
  & logger
  & freezeRequest

mainApp :: Application
mainApp req sendResponse = do
  body <- consumeRequestBodyStrict req         -- OK. request read
by main applicatoin
  ...

logger :: Middleware
logger app req f = do
  body <- consumeRequestBodyStrict req         -- OK. Request read
by logger
  ...
```

wai-extra
Hackage: https://hackage.haskell.org/package/wai-extra

Wai-extra
Middleware

```
--
module Network.Wai.Middleware.RequestLogger
logStdout :: Middleware
logStdoutDev :: Middleware
mkRequestLogger :: RequestLoggerSettings -> IO Middleware
...


--
module Network.Wai.Middleware.RequestLogger.JSON
requestToJOSN :: Request -> [ByteSTring] -> Maybe NominallDiffTime
-> Value


--
module Network.Middleware.Timetout
timeout :: Int -> Middleware
timeoutStatus :: Status -> Int -> Middleware
timeoutAs :: Response -> Int -> Middleware
```

==========

Warp (Web application)

Warp: https://hackage.haskell.org/package/warp-3.3.30
Warp performance: https://aosabook.org/en/posa/warp.html

Warp
Server handler for wai

Network.Wai.Handler.Warp

```
type Application = Request -> (Response -> IO ResponseReceived) ->
IO ResponseReceived
run :: Port -> Application -> IO ()
runEnv :: Port -> Application -> IO ()
            -- use PORT environment variable
runSettings :: Settings -> Application -> IO ()
runSettingsSocket :: Settings -> Socket -> Application -> IO ()

openFreePort :: IO (Port, Socket)
withApplication :: IO Application -> (Port -> IO a) -> IO a
            -- use open port
withApplicationSettings :: Settings -> IO Application -> (Port ->
IO a) -> IO a
testWithApplication :: IO Application -> (Port -> IO a) -> IO a


-- Settings
type Port = Int
data HostPreference
IsString HostPreference
```

```
data Settings
defaultSettings :: Settings
setPort :: Port -> Settings -> Settings
setHost :: HostPreference -> Settings -> Settings
setOnOpen :: (SockAddr -> IO Bool) -> Settings -> Settings
setOnClose :: (SockAddr -> IO ()) -> Settings -> Settings
setOnException :: (Maybe Request -> SomeException -> IO ()) ->
Settings -> Settings
setOnExceptionResponse :: (SomeException -> Response) -> Settings
-> Settings
setTimeout :: Int -> Settings -> Settings
setManager :: Manager -> Settings -> Settings
setInstallShutdownHandler :: (IO () -> IO ()) -> Settings ->
Settings
setLogger :: (Request -> Status -> Maybe Integer -> IO ()) ->
Settings -> Settings
setGracefulShutdownTimeout :: Int -> Settings -> Settings
...

getPort :: Settings -> Port
getHost :: Settings -> HostPreference
...


-- defaults
defaultOnException :: Maybe Request -> SomeException -> IO ()
defaultShouldDisplayException :: SomeException -> Bool
defaultOnExceptionResponse :: SomeException -> Response
exceptionResponseForDebug :: SomeException -> Response
```

usage: function to shutdown the server
Use **setInstallShutdownHandler**
Use **shared memory (TVar)** to store the handler

```
import Control.Concurrent
import Control.Concurrent.STM
import Control.Monad
import Control.Monad.IO.Class
import Data.ByteString.Lazy.Char8 qualified as B
import Data.Function
import Network.Wai
import Network.Wai.Handler.Warp

type ShutdownHandler = TVar (IO ())                    -- IO () is
the shutdown call

execShutdown :: ShutdownHandler -> IO ()
execShutdown = join . readTVarIO

shutdownTime :: Int
```

```
shutdownTime = 60         -- 60s

app :: Application
app = ...

main :: IO ()
main = do
  shutdownVar <- newTVarIO (pure ())                    -- Step 1)
create new handler
  let settings = createSettings shutdownVar

  _ <- forkIO $ runSettings settings app

  threadDelay $ shutdownTime * 1000 * 1000
  execShutdown shutdownVar                              -- Step 3)
call

createSettings :: ShutdownHandler -> Settings
createSettings shutdownVar =
  defaultSettings
    & setInstallShutdownHandler
        (atomically .writeTVar shutdownVar)             -- Step 2)
store shutdown handler
```

==========

```
This
is
code
Monaco 14
```